

AI Agent Architecture "Spectra" — BioSpecInfo

Field	Value
Project	BioSpecInfo — Spectra component (agentic AI copilot)
Author	Samuele Pio Provenzano
Thesis supervisor	Prof. Savino Longo — University of Bari Aldo Moro
Component	bsi-ai-hub.js — 4,895 lines, zero runtime dependencies
Type	Multi-provider conversational agent with client-side tool execution
Repository	samupropiol-ship-it/BioSpecInfo-v11
Documented version	Service Worker bsi-v133

1. Executive summary

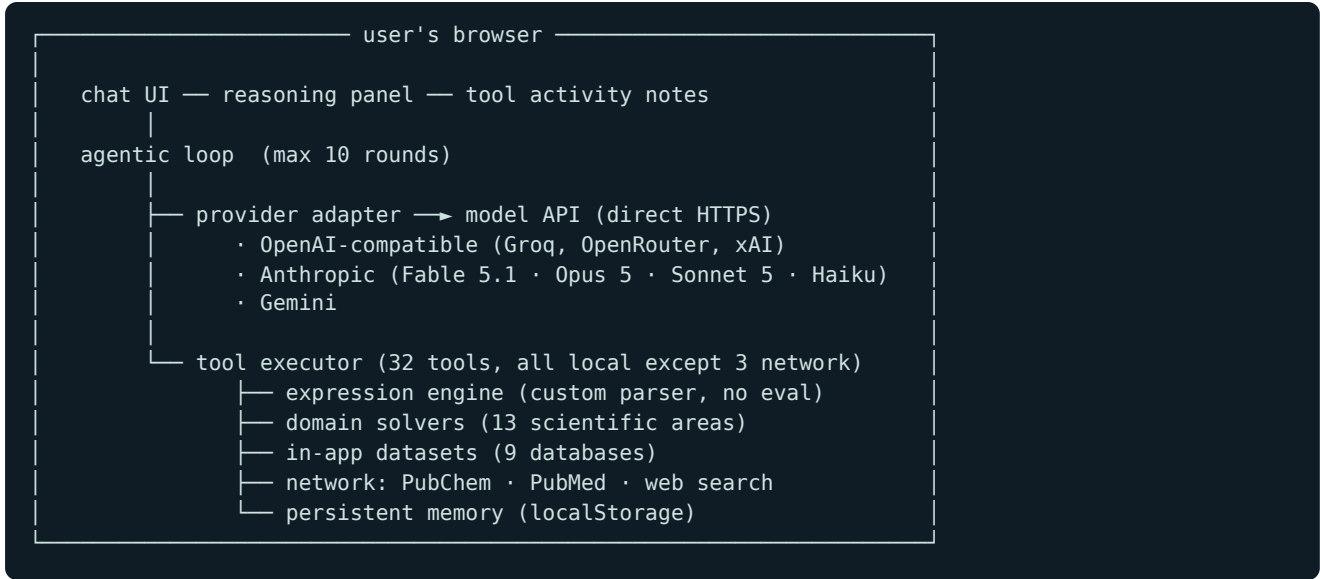
Spectra is an **agent**, not a thin wrapper around a language model. The difference is measurable: the model does not answer quantitative questions from memory — it **invokes deterministic tools** that compute the result, and the control loop lets it **chain up to 10 calls**, verifying each step before answering.

The architecture addresses three problems that separate a usable agent from a demo:

1. **Grounding** — a fabricated chemical value can be dangerous. 32 tools cover computation, public databases and the application's own datasets; the system prompt explicitly forbids quoting numeric values from memory.
1. **Transparency** — the model's reasoning is streamed live and stored alongside the answer, together with the list of tools actually invoked. The agent's work is auditable after the fact.
1. **Provider portability** — a single tool registry is translated into the three function-calling formats in use today (OpenAI-compatible, Anthropic, Gemini), so the agent behaves identically across eight model configurations, three of them free.

2. Architecture

2.1 Overview



There is no backend. API keys stay in the browser's `localStorage` and travel only to the selected provider: no data passes through BioSpecInfo servers, which do not exist.

2.2 Agentic loop

Each round: send the conversation history (last 40 turns) plus the system prompt and the schema of all 32 tools → receive the streamed response → if it contains tool calls, execute **all** of them → rebuild the assistant turn in the provider's native format → repeat.

Three exit conditions: no tool calls (final answer), `MAX_ROUNDS = 10` exhausted, or user interruption.

2.3 Provider-neutral tool registry

Tools are declared once in JSON Schema and translated at runtime:

Family	Required shape
Anthropic	<code>{name, description, input_schema}</code>
Gemini	<code>[{functionDeclarations: [...]}]</code>
OpenAI-compatible	<code>{type:"function", function:{...}}</code>

The same applies to messages: turns containing tool calls are serialised into each provider's native form and kept in a `_native` field, so a conversation stays coherent even when switching models.

3. The 32 tools

Area	Tools
General computation	calcola , risolv_i_equazione , analisi_dati
General chemistry	bilancia_equazione , stechiometria , massa_molecolare , converti_unita , costante_fisica
Physical chemistry	termodinamica , equilibrio_acido_base , cinetica , gas_e_soluzioni , elettrochimica
Structure & spectra	spettroscopia , quantistica_e_spettroscopia , cristallografia
Life sciences	biochimica , farmacocinetica , valuta_druglikeness
Physics	astrofisica , nucleare , statistica_inferenziale
External databases	cerca_pubchem (NIH), cerca_letteratura (PubMed), web search
Internal data	cerca_nel_database (9 datasets), cerca_molecola
App control	naviga_sezione (84 sections), apri_strumento (12 labs), stato_app
Memory	ricorda , ricordi
Animations	apri_animazione (6 reaction mechanisms)

3.1 Internal datasets exposed

297 synthesis reactions · 118 elements · 67 amino acids · 143 drugs · 63 pathologies · 39 retrosynthetic strategies · 36 drug interactions · 29 metabolic pathways · 29 redox potentials.

These are the datasets curated by the author for the application: the agent treats them as a primary source and is instructed to **report discrepancies** between the database and its own memory rather than silently smoothing them over.

4. Notable engineering decisions

This section documents the non-obvious choices and the reasoning behind them.

4.1 Expression engine without `eval()`

Problem. Serving arbitrary physical-chemistry computations requires an expression evaluator. `eval()` would solve it in a few lines.

Why it was rejected. The agent reads untrusted external content (PubChem results, PubMed abstracts, web pages). An injection in that content could induce the model to emit a hostile expression; with `eval()` that string would execute in the page context, **where the user's API key lives**.

Solution. Recursive-descent parser with a *closed* set of 27 functions and constants. It has no access to global scope: it can only do arithmetic. Validated against 10 escape attempts (`window.localStorage`, `constructor`,

`this`, `fetch(...)`, calls to non-whitelisted functions) — all rejected, including when executed in a real browser.

4.2 Preserving reasoning blocks

On models with adaptive reasoning, `thinking` blocks are part of the assistant turn and must be **echoed back unchanged, cryptographic signature included**, on the next tool-use round. Dropping them causes the request to be rejected.

The implementation accumulates `thinking_delta` and `signature_delta` from the stream and reinserts them **first** in the reconstructed turn. Verified in integration tests: signature preserved, ordering correct.

4.3 Resuming paused turns (`pause_turn`)

Web search is a server-side tool. When its internal loop exhausts its iterations, the response ends with `stop_reason: "pause_turn"`.

The turn must be resumed by **sending the assistant turn back as-is, without adding any user message**: the server recognises the trailing `server_tool_use` block and continues from there. Appending a "continue" message would break the mechanism.

4.4 Per-model configuration, not global

The four Claude tiers do not accept the same parameters:

Model	effort	Web search	Notes
Fable 5.1	<code>xhigh</code>	yes (8)	refusal fallbacks; reasoning always on
Opus 5	<code>high</code>	yes (6)	default
Sonnet 5	<code>high</code>	yes (5)	
Haiku 4.5	unsupported	unsupported	sending them returns HTTP 400

On all recent models `temperature`, `top_p` and `budget_tokens` were removed from the API and their presence causes an error: none of the four sends them. Every parameter is therefore conditioned on the provider rather than set globally.

4.5 Multimodal input and material reading

The agent accepts images, PDFs and text files, several at once, treated as consecutive pages of a single document. Three constraints shaped the implementation:

- **Images are downscaled to 2000 px** and re-encoded as JPEG before sending.

A phone photo drops from roughly 1 MB to 440 KB: without this step a single image would approach the request ceiling and multiply token cost. 2000 px is the minimum for reading small handwriting — at 1600 px fine script became illegible.

- **The PDF page ceiling depends on the model**, it is not a constant: 600

pages for models with a 1M-token window, 100 for 200K ones. Page counting is done by reading `/Type` `/Page` objects from the file bytes, without libraries.

- **History stores a 160 px thumbnail**, not the image that was sent. Storing

the large one (440 KB each) filled `localStorage` within a dozen photos, and once the quota is exceeded every subsequent write fails silently.

Six quick actions turn the material into a study artefact (summary, concept map, outline, flashcards, exam questions, transcription) and two translate it. For transcriptions the agent is instructed to write `[illegible]` rather than guess: an invented chemical term is worse than a declared gap.

4.6 In-house graph renderer, no CDN

Concept maps are produced by the model as Mermaid diagrams and drawn by a renderer written into the application (~190 lines): longest-path layering from the roots with cycle breaking, barycentre reordering over three passes to reduce edge crossings, SVG output with Bézier curves.

Not loading Mermaid from a CDN follows from the app's offline-first nature: a map must be drawable without a network. Unsupported syntaxes are recognised and left as a readable code block instead of being drawn badly.

4.7 Voice command with wake word

User-activated continuous listening: the phrase "Hey Spectra" followed by a command sends it automatically. Two non-obvious details:

- the browser's speech recognition **stops on its own** after a few seconds of silence and must be restarted, otherwise listening dies at the first pause;
- **all transcription alternatives** are examined, not just the first, and the most common phonetic variants are accepted: Italian recognition renders "Spectra" in many different ways.

Denying microphone permission stops the loop instead of retrying forever. The word "spettroscopia", extremely frequent in this domain, was verified as non-triggering.

4.8 Guard against non-finite results

A systematic audit revealed that, given legitimate degenerate inputs (zero wavelength, zero volume, zero half-life, a transition between identical levels), eight solvers returned `ok: true` with `Infinity` or `NaN` among their fields.

This is the most insidious failure in this context: an exception is noticed, a formally valid but meaningless value is reported to the user as correct. The fix is a single check at the point where all results pass through: if a numeric field is not finite, the result becomes an explicit error naming the fields and stating the probable cause. It also covers tools added in the future.

4.9 Graceful degradation

- **Models without reliable function calling** (some free tiers): on the first tool-related error the request is retried once in text-only mode instead of failing.
- **Idle timeout (45 s)**, reset on every byte received: a long answer is never truncated, but a dead connection is reported immediately with an explicit message.
- **localStorage quota**: history is capped (30 conversations, 100 messages)

with progressive pruning. Without this cap, exceeding the quota made every subsequent write fail *silently*, **including saving the API key**.

- **Classifier refusals** (`stop_reason: "refusal"` , HTTP 200): detected and surfaced, instead of appearing as an empty answer.

5. Verification

Verification was carried out at three levels, with recorded outcomes.

5.1 Numeric correctness — against reference values

Area	Case	Expected	Obtained
Balancing	<chem>KMnO4 + HCl -> ...</chem>	2:16:2:2:8:5	✓ exact
Molecular mass	<chem>K4[Fe(CN)6]</chem> (nested)	368.35	368.345
Molecular mass	<chem>CuSO4.5H2O</chem> (hydrate)	249.68	249.677
Equilibrium	acetic acid 0.1 M, K_a $1.8 \cdot 10^{-5}$	pH 2.874	2.875
Thermodynamics	ΔG (<chem>N2O4/NO2</chem>) at 298 K	+4.79 kJ/mol	✓
Kinetics	E_a from $k(300\text{ K})$, $k(320\text{ K})$	55.3 kJ/mol	55.33
Real gases	van der Waals <chem>CO2</chem>	−10 % deviation	−10.13 %
Spectroscopy	bromine M+2	97.3 %	✓
Biochemistry	glycylglycine	132.12 Da	✓
Pharmacokinetics	$t_{1/2}$ from V_d 50 L, Cl 5 L/h	6.93 h	✓
Astrophysics	solar emission peak (Wien)	501.5 nm	501.52
Astrophysics	Earth escape velocity	11.19 km/s	11.186
Nuclear	⁵⁶ Fe binding energy	8.79 MeV/nucleon	8.790
Statistics	critical t, 95 %, df = 4	2.7764	✓
Crystallography	copper density (fcc)	8.96 g/cm ³	8.935

P-values are not tabulated: they are computed with the regularised incomplete beta function (Lentz continued fraction) and the incomplete gamma function.

5.2 Robustness — cases that must fail

Verified as **rejected**: unbalanceable equations, formulas with unbalanced parentheses or non-existent elements, species absent from the equation, invalid database types, and the 10 injection attempts against the expression engine.

5.3 Integration — real browser

Automated tests with headless Chromium across all 14 application pages: no JavaScript errors, no missing resources. All 84 sections of `index.html` and the 18 tabs of the Astrochemistry module were traversed one by one. Request bodies were inspected for the four Claude tiers, and a full cycle with web search, turn suspension and resumption was simulated.

6. Quantitative summary

Metric	Value
Component size	4,895 lines
Tools	32
Model configurations	8 (3 free)
Scientific areas covered	13
Records in exposed internal datasets	over 800
Max agentic loop rounds	10
History sent to the model	40 turns
Runtime dependencies	none

7. Stated limitations

Documented for technical honesty; these are deliberate choices, not omissions.

- **The API key lives in the browser.** It is the only option without a backend: the key never leaves the device, but it is accessible to anyone with access to that browser. A shared key would require a server-side proxy with quota enforcement, outside the scope of a static application.
- **Network tools depend on service availability.** PubChem and PubMed are queried directly from the browser; when unreachable, the tool returns an explicit error and the agent is instructed to say so rather than substituting an estimate.
- **Solvers apply simplified models** where the literature offers more refined ones (e.g. Woodward–Fieser for UV, or ATP yields using average P/O ratios). Each tool states the method used in its own result.
- **Verification is functional and numeric**, not formal: there is no proof of solver correctness, but a set of reference cases drawn from the teaching literature.

*Companion document to the BioSpecInfo technical dossier. See also: [01-Software-Architecture-Document.md](#),

[02-Verification-Validation-Report.md](#), [03-Security-Privacy-Compliance.md](#).*